

Trabajo Práctico N°2 — Simulación de bandadas

Modelo de Vicsek y modelo de votante

Guía de estudio 2 de 4

Implementación

Diapositivas 5 a 7 • expositor B • 1:37

Arquitectura del motor y Cell Index Method

Grupo 5 — Comisión S

Lucas Di Candia • Juan Francisco Palermo • Gonzalo Sharif Curi Martinez

Documento interno de preparación de la defensa oral. No se entrega.

Índice

1. Qué se cuenta acá y qué no	2
2. Guion: diapositivas 5 a 7	2
3. El fondo: cada decisión de implementación y su justificación	3
3.1. Por qué $M = 9$ y no $M = 10$	3
3.2. Por qué la grilla se reconstruye en cada paso	3
3.3. Por qué el costo es $O(N)$ y no $O(N^2)$	4
3.4. Las dos condiciones periódicas, que son distintas	4
3.5. Medio vecindario: 5 celdas en vez de 9	4
3.6. La actualización sincrónica, en tres pasadas	4
3.7. La convención de la posición: se avanza con la velocidad vieja	5
3.8. El vecindario es autoinclusivo	5
3.9. Por qué $\arctan 2$ de las sumas y no un promedio de ángulos	5
3.10. Por qué se renormaliza el ángulo a $[-\pi, \pi]$	5
3.11. El generador pseudoaleatorio	6
3.12. Compilación	6
4. Preguntas y respuestas	6
4.1. Sobre el Cell Index Method	6
4.2. Sobre la actualización	7
4.3. Sobre el código y la comparación entre modelos	7

1. Qué se cuenta acá y qué no

La guía de la cátedra (punto 2.2) delimita esta sección con precisión:

“Detalle de cómo se traduce el modelo matemático al modelo computacional, describiendo arquitectura del código implementado, pseudocódigo, diagrama UML, etc. **Solo considerar el motor de simulación propiamente dicho, dejando fuera de esta descripción cómo se implementa el post-proceso, o el formato de los archivos input/output, etc.**”

Cuidado

No hables del formato de los archivos de texto, ni de `sweep.py`, ni de `analyze.py`, ni de cómo se hacen las animaciones. Está explícitamente excluido. Si te preguntan por el output, contestás en el turno de preguntas (está en el cuadernillo 3), pero no lo pongas en el guion.

Tenés dos diapositivas de contenido y 1:37 en total. Es el bloque más corto: sé quirúrgico.

2. Guion: diapositivas 5 a 7

Diapositiva 5 • Divisoria: Implementación

2 s • expositor B

Solo el título de la sección. Es tu señal de entrada.

Diapositiva 6 • Arquitectura del motor

50 s • expositor B

A la izquierda el diagrama de flujo del paso temporal; a la derecha cuatro puntos. Recorré el diagrama de arriba hacia abajo y cerrá con la sincronía.

“El motor está escrito en C++20, sin dependencias externas. La clase `Simulation` concentra todo el estado: las partículas, la grilla y el generador pseudoaleatorio. La partícula guarda solamente su posición y su ángulo: la velocidad no se almacena, se deriva del ángulo, porque el módulo es constante. Cada paso hace tres cosas: primero reconstruye la grilla del `Cell Index Method` para obtener la lista de vecinos; después calcula las orientaciones nuevas con la regla de alineación, que es lo único que distingue Vicsek del votante; y por último integra las posiciones aplicando las condiciones periódicas. Los dos modelos comparten absolutamente todo el código salvo esa función de alineación. Y algo importante: la actualización es sincrónica, las orientaciones nuevas se escriben en un buffer aparte, así que ninguna partícula ve el ángulo nuevo de otra dentro del mismo paso.”

Lo que tenés que poder decir

Los dos puntos que no podés dejar de decir: (1) los dos modelos comparten todo el motor menos una función, y (2) la actualización es sincrónica gracias a un buffer auxiliar. El primero es lo que hace legítima la comparación de la sección de resultados; el segundo es lo que hace que esto sea un autómata celular bien planteado.

Diapositiva 7 • Búsqueda de vecinos: Cell Index Method

45 s • expositor B

A la izquierda el esquema de la grilla con la celda central resaltada y la condición $L/M > r_c$; a la derecha cuatro puntos.

“Para la búsqueda de vecinos reutilizamos el `Cell Index Method` del TP1. Dividimos la caja en una grilla de M por M celdas, con el lado de celda elegido de forma que sea mayor que el radio de interacción. Esa condición garantiza que todos los vecinos posibles de una partícula estén en su propia celda o en las ocho adyacentes, así que solo comparamos contra ese vecindario en lugar de contra las N partículas. Con las condiciones periódicas, la última fila de celdas es

adyacente a la primera, así que los índices se cierran sobre el toro. El costo pasa de orden N^2 a orden N . Y como las partículas se mueven, la grilla se reconstruye en cada paso.”

Lo que tenés que poder decir

La frase clave es “*el lado de celda tiene que ser mayor que el radio de interacción*”: es la condición que hace correcto al método. Si el lado fuera menor, habría vecinos afuera del bloque de 3×3 y se perderían.

Cuidado

La diapositiva dice “sólo se comparan las 9 celdas vecinas”. Eso es correcto *conceptualmente*, pero el código usa la optimización de medio vecindario: recorre 5 celdas por celda y registra cada par encontrado en las listas de las *dos* partículas. El resultado es idéntico al de recorrer las 9, con la mitad del trabajo. Si te preguntan, tenés que saberlo (ver P6).

Cerrá pasándole la palabra a C:

“Con el motor descripto, $[C]$ pasa al sistema concreto que simulamos.”

3. El fondo: cada decisión de implementación y su justificación

Esta es la sección donde más pueden pinchar, porque cada línea de código es una decisión que hay que poder defender. Van ordenadas por probabilidad de que las pregunten.

3.1. Por qué $M = 9$ y no $M = 10$

El número de celdas por lado lo calcula `maxValidGridM(L, rc)`: devuelve el mayor entero M tal que el lado de celda L/M sea *estrictamente* mayor que r_c .

$$M = \max(1, m), \quad m = \begin{cases} \lfloor L/r_c \rfloor - 1, & \text{si } L/r_c \text{ es entero,} \\ \lfloor L/r_c \rfloor, & \text{en caso contrario.} \end{cases}$$

Con $L = 10$ y $r_c = 1$ el cociente da exactamente 10, que es entero, así que se resta uno y queda $M = 9$, con lado de celda $10/9 \approx 1,11 > 1$.

El número

Si tomáramos $M = 10$, el lado de celda sería exactamente $r_c = 1$. Dos partículas separadas por una distancia apenas menor que r_c —o sea, vecinas— podrían caer en celdas *no* adyacentes, y el método las perdería. La desigualdad tiene que ser estricta, y por eso se resta uno cuando el cociente es entero. Buscar más celdas de las válidas no acelera nada: rompe el método.

3.2. Por qué la grilla se reconstruye en cada paso

Porque las partículas se mueven. En un autómata celular de red el vecindario es fijo y se calcula una vez; acá, después de integrar las posiciones, una partícula puede haber cambiado de celda y su conjunto de vecinos es otro. No reconstruir significaría usar una lista de adyacencia desactualizada.

Ahora bien, *reconstruir* no significa *reasignar memoria*. La clase `Grid` es un objeto persistente: los vectores de celdas y de listas de vecinos se *vacían* y se vuelven a llenar en cada llamada, sin liberar ni volver a reservar. Eso importa porque el barrido paramétrico completo hace del orden de 10^6 reconstrucciones.

Por qué es así

Esa es la diferencia principal con el `computeCIM` del TP1, que creaba y destruía sus buffers en cada llamada. Es también parte de por qué en el benchmark (diapositiva 25) el CIM de TP2 sale *más rápido* que el de TP1 para el mismo N .

3.3. Por qué el costo es $O(N)$ y no $O(N^2)$

Con fuerza bruta se comparan todos los pares: $N(N - 1)/2$ comparaciones. Con el CIM, cada partícula solo se compara contra las que están en su celda y en las adyacentes. Si la densidad se mantiene constante, el número esperado de partículas por celda es constante, así que cada partícula hace un número *constante* de comparaciones y el total escala como N .

Cuidado

La linealidad vale a **densidad constante**. En nuestro benchmark aumentamos N con la caja fija en $L = 10$, así que la densidad crece con N y hay más partículas por celda: el escalado medido resulta superlineal. Esto lo explica C en la diapositiva 25, pero conviene que lo sepas porque te lo pueden preguntar acá.

3.4. Las dos condiciones periódicas, que son distintas

La periodicidad interviene en dos lugares separados, y conviene no confundirlos:

1. **En la indexación de celdas.** Los índices de celdas vecinas se cierran sobre el toro con $\text{wrap}(i, M) = ((i \bmod M) + M) \bmod M$, de modo que la última fila de celdas es adyacente a la primera.
2. **En el cálculo de la distancia.** El predicado de vecindad aplica la *convención de imagen mínima*: $\Delta = d - L \text{round}(d/L)$ a cada componente, antes de comparar contra r_c^2 . Sin eso, dos partículas a ambos lados del borde darían una distancia enorme en lugar de la correcta.

El constructor de Grid exige además $L/2 > r_c$. Esa condición garantiza que la imagen mínima sea *única*: si r_c fuera mayor que media caja, una partícula podría ser vecina de otra por dos caminos distintos alrededor del toro y la convención dejaría de estar bien definida. Con $L = 10$ y $r_c = 1$ se cumple holgadamente.

3.5. Medio vecindario: 5 celdas en vez de 9

Cuando la grilla tiene $M \geq 3$, el código recorre solo 5 de las 9 celdas del vecindario: la propia, y las de arriba, derecha, arriba-derecha y arriba-izquierda. Al encontrar un par (i, j) lo registra en las listas de *ambas* partículas.

El resultado es idéntico al de recorrer las 9 —cada par se descubre exactamente una vez, desde uno de los dos lados— pero con la mitad de las comparaciones. Es la misma optimización que ya traía el CIM del TP1.

Por qué es así

La condición $M \geq 3$ no es caprichosa. Con $M < 3$ y contorno periódico, el *wrap* hace que una misma celda aparezca dos veces en el stencil de medio vecindario y el par se registraría duplicado; por eso el código cae al vecindario completo de 9 celdas en ese caso. Con $M = 9$ estamos siempre en el camino optimizado.

3.6. La actualización sincrónica, en tres pasadas

El método `step()` hace exactamente esto:

1. **Reconstruye la grilla** sobre las posiciones del instante t .
2. **Calcula todas las orientaciones nuevas** leyendo únicamente el estado viejo, y las deposita en un buffer aparte, `thetaNew_`. El campo `theta` de las partículas no se escribe en ningún punto de este bucle.
3. **Integra las posiciones** y recién ahí confirma los ángulos nuevos.

Lo que tenés que poder decir

Si no hubiera buffer auxiliar y se escribiera θ_i en el mismo bucle, la partícula $i + 1$ vería el ángulo *nuevo* de la partícula i dentro del mismo paso. Eso sería una actualización *asincrónica*, el resultado dependería del orden de recorrido de las partículas —que es arbitrario— y dejaría de ser un autómata celular bien definido. Es la razón de ser del buffer.

3.7. La convención de la posición: se avanza con la velocidad vieja

Este punto es sutil y conviene tenerlo claro porque se corrigió después de la clase de consultas. La integración es

$$\mathbf{x}_i(t + \Delta t) = \mathbf{x}_i(t) + \mathbf{v}_i(t) \Delta t,$$

con $\mathbf{v}_i(t)$ derivada de $\theta_i(t)$, o sea del ángulo **viejo**, tal como lo escribe la diapositiva 42 de la Teórica 2. En el código: la tercera pasada calcula el desplazamiento con `p.theta` (todavía el viejo) y *después* asigna `p.theta = thetaNew_[i]`.

El número

La consecuencia física es que el ruido sorteado en el paso t recién afecta el desplazamiento del paso siguiente: la partícula primero avanza en la dirección que traía y después gira. La convención opuesta (girar y después avanzar) da un sistema sistemáticamente menos polarizado; la diferencia en v_a llegaba a 0,10 en la región de transición. Usamos la de la teórica.

3.8. El vecindario es autoinclusivo

$\mathcal{N}_i$ contiene a la propia partícula i además de sus vecinos. En Vicsek, la suma de senos y cosenos arranca con el θ_i propio antes de sumar los vecinos —es la convención del paper original—. En el votante, el sorteo se hace sobre $\{i\} \cup \mathcal{N}_i$, así que una partícula puede “votarse a sí misma”.

Esto resuelve un caso de borde real: **¿qué hace una partícula que no tiene ningún vecino dentro de r_c ?** Con la convención autoinclusiva, conserva su propia orientación más el ruido, y la regla queda bien definida. Sin ella, el promedio circular sería 0/0 y el sorteo del votante sería sobre un conjunto vacío.

El número

No es un caso marginal. En el estudio de clusters a densidades subcríticas ($N = 11$ a 32 , con $\langle k \rangle < 1$) las partículas aisladas son frecuentes, y la convención afecta una fracción no despreciable de las actualizaciones.

Cuidado

En el código, el sorteo del votante es `uniform_int_distribution<size_t> pick(0, row.size())`: el índice igual a `row.size()` representa a la propia partícula. Escribirlo como `(0, row.size()-1)` sobre un vecindario vacío daría *comportamiento indefinido* por desbordamiento del entero sin signo. Está hecho a propósito así.

3.9. Por qué atan2 de las sumas y no un promedio de ángulos

Si promediáramos los ángulos directamente, dos partículas a $+179^\circ$ y -179° —que apuntan casi en la misma dirección— darían un promedio de 0° , o sea la dirección opuesta. Es la discontinuidad de la variable angular en $\pm\pi$.

La solución es el *promedio circular*: se suman los senos y los cosenos por separado —o sea, se suman los vectores unitarios— y se recupera el ángulo con `atan2` de esas dos sumas. Eso es geoméricamente correcto en toda la circunferencia.

3.10. Por qué se renormaliza el ángulo a $[-\pi, \pi]$

Después de sumar el ruido, el código hace $\theta \leftarrow \text{atan2}(\sin \theta, \cos \theta)$, que devuelve el mismo ángulo pero llevado al intervalo $[-\pi, \pi]$.

En Vicsek sería redundante, porque `atan2` ya devuelve un valor en rango. Pero en el votante es **necesario**: la partícula copia *textualmente* el ángulo ya comprometido de otra, en lugar de recalcularlo. Sin la renormalización, el ángulo acumularía una caminata aleatoria no acotada —sumando ruido sobre ruido a lo

largo de miles de pasos— y crecería sin límite en valor numérico, con la consiguiente pérdida de precisión en los senos y cosenos.

3.11. El generador pseudoaleatorio

Es un `std::mt19937_64` (Mersenne Twister de 64 bits) sembrado explícitamente, nunca desde el reloj. El ruido de los dos modelos pasa por la *misma* función `addAngularNoise`, así que ambos usan exactamente la misma distribución y el mismo flujo de números aleatorios. Cómo se derivan las semillas del barrido lo cuenta C (cuadernillo 3).

3.12. Compilación

C++20, compilado con `-O2 -Wall -Wextra -pedantic`, sin dependencias externas ni CMake: un Makefile plano. Hay además un binario de autotest (`tp2_test`) que verifica invariantes del CIM — simetría de la lista de adyacencia, ausencia de auto-vecinos y de duplicados— contra una fuerza bruta de referencia.

El número

El `-O2` no es cosmético: las mediciones de tiempo del inciso (g) carecen de sentido sin optimización.

4. Preguntas y respuestas

4.1. Sobre el Cell Index Method

P1. ¿Por qué $M = 9$ y no $M = 10$, si $L = 10$ y $r_c = 1$?

R. Porque el lado de celda tiene que ser *estrictamente* mayor que r_c . Con $M = 10$ el lado sería exactamente 1, y dos partículas separadas por una distancia apenas menor que r_c podrían caer en celdas no adyacentes y perderse como vecinas. Por eso la fórmula resta uno cuando L/r_c da entero: queda $M = 9$ y el lado de celda es $10/9 \approx 1,11$.

P2. ¿Qué pasaría si usaran un M más grande? ¿No sería más rápido?

R. No: sería incorrecto. El método deja de encontrar todos los vecinos. El constructor de la grilla directamente lanza una excepción si le pasás un M mayor que el máximo válido.

P3. ¿Por qué reconstruyen la grilla en cada paso? ¿No es caro?

R. Es necesario, porque las partículas se mueven y cambian de celda. Lo que sí evitamos es el costo de reservar memoria: la grilla es un objeto persistente cuyos vectores se vacían y se vuelven a llenar, sin liberar ni volver a reservar. Sobre el orden de 10^6 reconstrucciones que hace el barrido completo, eso es lo que importa.

P4. Dicen “9 celdas”, pero ¿el código recorre las 9?

R. No, recorre 5 y registra cada par en las listas de las dos partículas. El resultado es idéntico al de recorrer las 9 —cada par se descubre una sola vez, desde uno de los dos lados— con la mitad de las comparaciones. Es la optimización de medio vecindario, la misma del TP1. Conceptualmente el vecindario sigue siendo el bloque de 3×3 .

P5. ¿Cómo manejan las condiciones periódicas en el CIM?

R. En dos lugares distintos. En la indexación, los índices de celda se cierran sobre el toro con una operación de módulo, así que la última fila es adyacente a la primera. Y en la distancia, aplicamos la convención de imagen mínima antes de comparar contra el radio. Además exigimos $L/2 > r_c$ para que la imagen mínima sea única; con $L = 10$ y $r_c = 1$ se cumple de sobra.

P6. ¿El CIM es realmente $O(N)$?

R. A densidad constante, sí: el número esperado de partículas por celda es constante, así que cada partícula hace un número constante de comparaciones. En nuestro benchmark aumentamos N con la caja fija, así que la densidad crece y el escalado medido da superlineal. Eso está discutido en la diapositiva 25.

P7. ¿Verificaron que el CIM da lo mismo que la fuerza bruta?

R. Sí, hay un binario de autotest que compara la lista de adyacencia del CIM contra una fuerza bruta de referencia y verifica los invariantes: simetría, que ninguna partícula sea vecina de sí misma y que no haya duplicados.

4.2. Sobre la actualización**P8. ¿Por qué la actualización es sincrónica? ¿Qué cambiaría si no lo fuera?**

R. Las orientaciones nuevas se escriben en un buffer aparte y recién se confirman al final del paso, así que ninguna partícula ve el ángulo actualizado de otra dentro del mismo paso. Si fuera asincrónica, el resultado dependería del orden en que se recorren las partículas, que es arbitrario, y el modelo dejaría de estar bien definido como autómata celular.

P9. ¿La posición se actualiza con la velocidad vieja o con la nueva?

R. Con la vieja: $\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t) \Delta t$, tal como está en la diapositiva 42 de la Teórica 2. La partícula primero avanza en la dirección que traía y después gira, así que el ruido sorteado en un paso recién afecta el desplazamiento del paso siguiente.

P10. ¿Qué hace una partícula que se queda sin vecinos?

R. El vecindario es autoinclusivo: contiene a la propia partícula. En Vicsek, el promedio circular arranca sumando su propio seno y coseno —es la convención del paper original—, así que una partícula aislada conserva su orientación más el ruido. En el votante, el sorteo se hace sobre el conjunto que incluye a la propia partícula, así que se “vota a sí misma”. En los dos casos la regla queda bien definida. No es un caso marginal: en el estudio de clusters a densidades subcríticas las partículas aisladas son frecuentes.

P11. ¿Por qué usan `atan2` de las sumas de senos y cosenos en vez de promediar los ángulos?

R. Porque promediar ángulos directamente rompe en $\pm\pi$: dos partículas a $+179^\circ$ y -179° apuntan casi igual, pero su promedio aritmético da 0° , la dirección opuesta. Sumando los senos y cosenos por separado estamos sumando vectores unitarios, que es lo geoméricamente correcto, y recuperamos el ángulo con `atan2`.

P12. ¿Por qué renormalizan el ángulo a $[-\pi, \pi]$?

R. Por el modelo de votante. Ahí la partícula copia textualmente un ángulo ya comprometido en vez de recalcularlo con `atan2`, así que sin renormalizar el ángulo acumularía una caminata aleatoria no acotada a lo largo de miles de pasos. En Vicsek sería redundante porque `atan2` ya devuelve un valor en rango, pero usamos la misma función en los dos para que el tratamiento del ruido sea idéntico.

4.3. Sobre el código y la comparación entre modelos**P13. ¿Cuánto código comparten los dos modelos?**

R. Todo salvo una función. Hay un `enum` que selecciona entre `circularMeanHeading` y `voterHeading`; el resto —la grilla, el ruido, la integración, las condiciones periódicas, los observables— es el mismo código. El ruido de los dos pasa por la misma función, con la misma distribución y el mismo flujo de números

aleatorios. Eso es lo que hace que la comparación mida la diferencia entre las reglas y no diferencias de implementación.

P14. ¿Por qué la partícula no guarda la velocidad?

R. Porque el módulo es constante por construcción: la velocidad queda determinada por el ángulo. Guardar las dos componentes sería estado redundante que habría que mantener consistente. Se derivan cuando hacen falta, con seno y coseno.

P15. ¿Qué generador de números aleatorios usan?

R. Mersenne Twister de 64 bits, sembrado explícitamente, nunca desde el reloj. Todo el barrido es reproducible bit a bit.

P16. ¿Por qué C++ y no Python para el motor?

R. Por el volumen del barrido: son 468 puntos de simulación por 10 semillas, con corridas de 2000 pasos —y de 10 000 en las densidades subcríticas—, del orden de 10^6 reconstrucciones de grilla. En Python no cerraba. El análisis y los gráficos sí están en Python, sobre los archivos de texto que escribe el motor.

P17. ¿Reutilizaron el código del TP1?

R. Sí, el Cell Index Method. Lo reorganizamos en una clase Grid persistente en vez de la función que reasignaba sus buffers en cada llamada, y le sacamos el radio de las partículas, porque acá son puntuales. La lógica de indexación, el *wrap* periódico y la imagen mínima son las mismas.